

Θεωρία: Truthy vs Falsy, Εξαιρέσεις, Συναρτήσεις



Truthy vs Falsy

Γενικά, η python μας επιτρέπει να αλλάξουμε τον τύπο των δεδομένων μας και άρα τον τρόπο με τον οποίο τα χειριζόμαστε.

Παράδειγμα: `int(input("price:"))` -> μετατροπή από string σε int

Τι γίνεται όταν θέλουμε να χρησιμοποιήσουμε μια τιμή ως bool?



Όλα τα αντικείμενα στη python χωρίζονται σε Truthy και Falsy. Αυτό σημαίνει ότι

- αν χρησιμοποιήσουμε την εντολή `bool()`
- ή αν βάλουμε το αντικείμενο σε boolean context, δηλαδή σε κάποιο σημείο που η python περιμένει να αποτιμήσει μια συνθήκη ως True ή False π.χ. μετά από ένα if ή ένα while,

το αντικείμενο θα αντιστοιχηθεί σε μία τιμή True ή False.

- Όλοι οι αριθμοί είναι Truthy, εκτός από το 0.
- Τα ακολουθιακού τύπου δεδομένα της python χωρίζονται σε Truthy και Falsy με βάση το μήκος τους. Αντί να αξιολογούμε το ίδιο το δεδομένο, αξιολογούμε το μήκος του, ως αριθμό. Ακολουθιακού τύπου δεδομένα είναι οι συμβολοσειρές (strings), οι λίστες κ.α.
- Το ίδιο συμβαίνει και με άλλους τύπους δεδομένων που θα δούμε στη συνέχεια που αποτελούν "συλλογές" (collections): τα σύνολα (sets) και τα λεξικά (dictionaries)



Εξαιρέσεις - Exceptions

Όταν, κατά την εκτέλεση του, ένα πρόγραμμά μας οδηγείται σε μια λειτουργία που παραβιάζει τους κανόνες της python, βλέπουμε στον υπολογιστή μας κάποιο μήνυμα εξαίρεσης (exception). Το exception περιέχει χρήσιμη πληροφορία που μας επιτρέπει να καταλάβουμε "τι πήγε στραβά". Μπορεί να ανήκει σε μία από τις παρακάτω κατηγορίες:

Κατηγορία	Σημασία	Παράδειγμα
TypeError	Ο τύπος των δεδομένων δεν είναι συμβατός με τη ζητούμενη λειτουργία	<code>"Number: " + 5</code>
ValueError	Η τιμή δεν είναι συμβατή με τον τύπο των δεδομένων	<code>int("letters")</code>
ZeroDivisionError	Ο δεύτερος αριθμός σε μια διαίρεση ή ένα υπόλοιπο (modulo) είναι μηδενικός	<code>100/0</code>
IndexError	ο δείκτης που προσπαθούμε να προσπελάσουμε είναι εκτός των ορίων της λίστας	<code>my_list = [1, 2, 3]</code> <code>print(my_list[100])</code>

Εάν υποψιαζόμαστε πως μπορεί να συμβεί κάτι τέτοιο (π.χ. γιατί περιμένουμε input από χρήση που δεν εμπιστευόμαστε...) μπορούμε να τρέξουμε τον κώδικα μας σε ένα δοκιμαστικό πλαίσιο. Εάν

προκύψει μια εξαίρεση, λέμε στο πρόγραμμα μας πως να τη διαχειριστεί. Δηλαδή, τι να κάνει αν η λειτουργία που ζητήσαμε αρχικά δεν επιτρέπεται. Η διαδικασία αυτή ονομάζεται **Exception Handling**.

! ΠΡΟΣΟΧΗ: Το πρόγραμμα μας δε θα μπορέσει ποτέ να εκτελέσει την απαγορευμένη λειτουργία. Εμείς το βάζουμε να ακολουθήσει έναν εναλλακτικό δρόμο για να μην “κρασάρει”.

Συνταξη:

- Για να διαχειριστούμε όλα τα exceptions :

```
try:
    #επικίνδυνος κώδικας
except :
    #τι να κάνουμε σε περίπτωση exception
else:
    #τι να κάνουμε αν δεν γίνει exception
```

- Για να διαχειριστούμε ένα συγκεκριμένο exception, έστω SomeException :

```
try:
    #επικίνδυνος κώδικας
except SomeException:
    #τι να κάνουμε σε περίπτωση συγκεκριμένου exception
else:
    #τι να κάνουμε αν δεν γίνει το συγκεκριμένο exception
```

! ΠΡΟΣΟΧΗ: Σε αυτή την περίπτωση, δε διαχειριζόμαστε τα υπόλοιπα exceptions που μπορεί να προκύψουν.

Μπορούμε στον κώδικα μας να διαχειριζόμαστε με άλλο τρόπο διαφορετικά είδη exceptions, όπως φαίνεται στο παρακάτω παράδειγμα.

```
try:
    # Risky operation: dividing string by number
    res = "100" / 20

except ArithmeticError:
    print("Arithmetic problem.")

except:
    print("Something went wrong!")
```

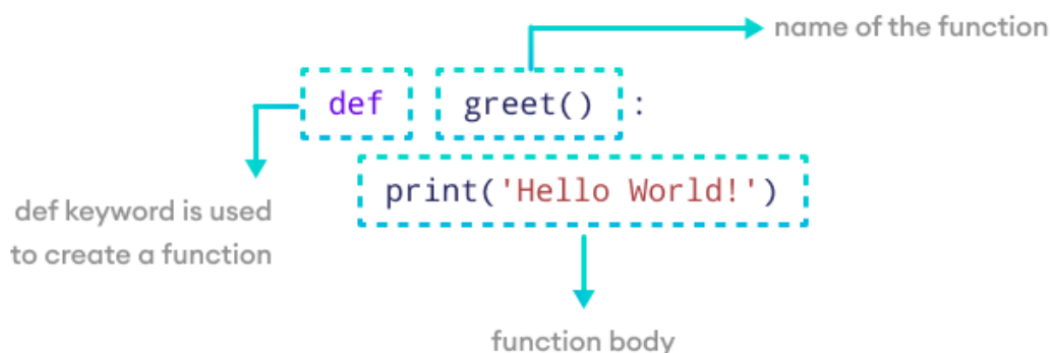
Τι θα τυπωθεί εδώ;;



Functions (Συναρτήσεις)

Πολλές φορές, θέλουμε να επαναχρησιμοποιούμε κομμάτια κώδικα στα προγράμματά μας. Οι συναρτήσεις είναι ένας τρόπος να οργανώνουμε τον κώδικα μας και να γλιτώνουμε χρόνο πληκτρολόγησης!

Σύνταξη:



- Η εντολή `def` δηλώνει ότι ορίζουμε συνάρτηση
- το όνομα της συνάρτησης ακολουθείται από παρένθεση
- ο κώδικας που εκτελείται όταν θα **καλούμε τη συνάρτηση**, **δηλαδή όταν εκτελούμε τον κώδικα που περιέχει** γράφεται μετά από την άνω-κάτω τελεία και πιο μέσα, ακριβώς όπως στο σώμα του `if`!

Για να καλέσουμε τη συνάρτηση στο πρόγραμμά μας, απλώς γράφουμε `greet()` σε μία καινούρια γραμμή, αφού ορίσουμε τη συνάρτηση!

Ο κώδικας εντός μιας συνάρτησης εκτελείται μόνο αν την καλέσουμε!!!

Για ποιο λόγο, όμως, υπάρχει η παρένθεση δίπλα στο όνομα της συνάρτησης;



Σε πολλές περιπτώσεις περνάμε **ορίσματα** στα `functions` μας. Τα ορίσματα βοηθάνε όταν θέλουμε ο κώδικας της συνάρτησης να μπορεί να χρησιμοποιήσει κάποιο επιπλέον δεδομένο.

Οι μεταβλητές που παίρνουν τα ορίσματα λέγονται **παράμετροι**.

Παράδειγμα	Έξοδος
<pre>def my_function(number): number = number / 2 print(number) my_function(10) x = 20 my_function(x)</pre>	<pre>5.0 10.0</pre>

- Σαν όρισμα μπορεί να μπει απευθείας ένα δεδομένο ή μια μεταβλητή.
- Οι παράμετροι λέγονται τοπικές μεταβλητές γιατί ζουν μόνο μέσα στη συνάρτηση. Δεν είναι ορατές στο κύριο πρόγραμμά μας. Το ίδιο ισχύει και για όλες τις άλλες μεταβλητές που μπορεί να ορίσουμε εντός της συνάρτησης.

Μπορούμε να ορίσουμε τις συναρτήσεις μας με τρόπο ώστε να επιστρέφουν κάποια τιμή. Για να το κάνουμε αυτό γράφουμε `return` στο τέλος τους και δίπλα την τιμή που θέλουμε να επιστρέψουν.

Παράδειγμα	Έξοδος
<pre>def my_function(number): number = number / 2 return number x = my_function(10) print(x + my_function(20))</pre>	15.0

Μπορείτε να σκέφτεστε τη συνάρτηση σαν μια αριθμητική πράξη!